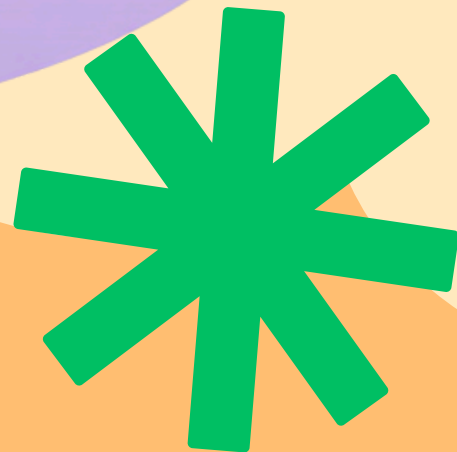


FUN WITH PYTHON: GENERATORS & ITERATORS

Learning how Python helps
us run through data



WHAT IS AN ITERATOR?

An iterator is like a magical bookmark that helps us go through a list one item at a time.

- It helps computers remember where they are in a list.
- Think of it like turning pages in a book – you only see one page at a time!
- Iterators make it easy to visit every item without getting lost.

In our coding adventure, we'll learn how to use these helpful tools to work with lists of numbers, words, and more!

WHY DO WE USE ITERATORS?

Iterators are super helpful tools that make working with data much easier and more organized. They help us focus on one thing at a time instead of getting overwhelmed by big lists of information.



To look at one thing at a time

To not get lost in big lists

To save time and energy

HOW DOES ITERATION WORK IN PYTHON?

In Python, we use loops to go through each item in a list. Think of it like reading a story one page at a time!

```
fruits = ['apple', 'banana', 'mango']  
for fruit in fruits:  
    print(fruit)
```

This loop shows each fruit in the list, one by one!

LOOPING THROUGH LISTS STEP-BY-STEP

- Watch how the loop goes through each fruit one at a time!
- Step 1: The loop starts with fruit = 'apple' and prints it first.
- Step 2: Now the loop moves to fruit = 'banana' and prints it second.
- Step 3: Finally, the loop reaches fruit = 'mango' and prints it last.
- The loop keeps going until there are no more items left in the list!



WHAT IS A GENERATOR?

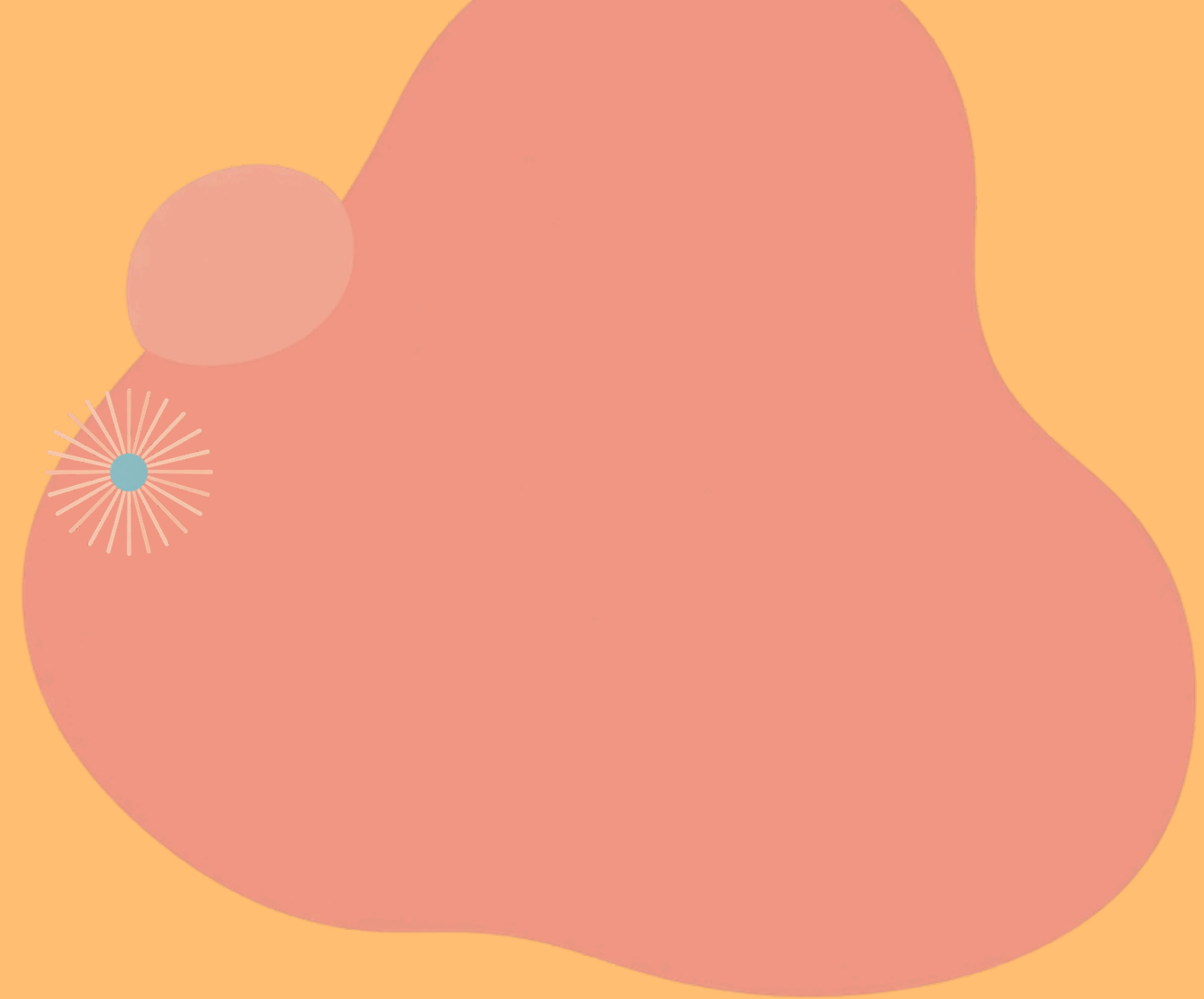
- A generator is like a magic machine that creates items one at a time when you ask for them.
- Think of a toy factory! When you press a button, it makes just one toy. Press again, and it makes another. It doesn't make all the toys at once – that would take up too much space!
- This is cool because:
 - It saves space because it doesn't make all items at once.
 - It's efficient – only creates what you need, when you need it.
 - It's perfect for big lists of things!

HOW GENERATORS WORK IN PYTHON

- Here's a simple generator that counts to three:
- `def count_up_to_three():`
 - `yield 1`
 - `yield 2`
 - `yield 3`
 -
- `for number in count_up_to_three():`
 - `print(number)`
- The 'yield' word is special! It sends one item out at a time, pausing the function and waiting for the next request. This is how generators work in Python!

ITERATORS VS GENERATORS

Understanding the difference between iterators and generators helps us write smarter, more efficient Python code.
Think of them as two different ways to go through data!



Iterators:
Go through items one by one



Iterators:
Can be created from lists



Generators:
Make items only when needed



Generators:
Use 'yield' to give items



**THANKS FOR
LEARNING
WITH US!**

